

R Bootcamp 1 - Introduction to R

Giovanni Castro Irizarry
UCLA M183/M136C

Spring 2025 Week 2

Housekeeping

First of all, throughout any session, stop me to ask questions! If you don't get something, chances are most others don't get it either. Do not be shy. Interrupt me. I will miss things. Also, I encourage you to help each other. If you finish with a mini-assignment, turn to your neighbor and see if they need help or are stuck. Teaching helps you better understand, too.

Please note that time was not enough to cover all of this Bootcamp, so please take your time to review and understand the sections, commands, and concepts that we did not review in section.

I borrowed and adapted a lot of the material in this course from Tyler Reny at Claremont Graduate University <http://tylerreny.github.io/> and previous TA's for this course.

Before we get started, did everyone install R and RStudio?

- If not, spend the next few minutes downloading them
- Rstudio (<https://www.rstudio.com/products/rstudio/download2/>)
- R (<https://www.r-project.org/>)

How do I navigate R Studio?

4 quadrants

1. Top Left - where you code (where you do your work)
2. Top Right - Environment and other "useful" things (don't worry about this area)
3. Bottom Right - Files and Plots generated here (where you look at the work you've done)
4. Bottom Left - Console/Terminal (you can code in the console, but don't)

R Markdown

- This is R Markdown
- A file format that allows you to create documents that serve as a neat record of your analysis
- File > New File > R Markdown > Create "Title" and "Author" > Choose "PDF" for "Default Output Format"
- You can change the 'title' at the very top
- You can save the document
- To learn more about R Markdown and formatting:
 - <https://www.stat.cmu.edu/~cshalizi/rmarkdown/>
 - <https://www.rstudio.com/wp-content/uploads/2015/02/rmarkdown-cheatsheet.pdf>

Running code

- You run code that you have written.
 - You can run your code by:
 - * Clicking “Run” in the top right of the Top Left quadrant
 - * Hitting “Ctrl + Enter” (on PC)
 - * Hitting “Command + Return” (on Mac)

Knit

- You can “Knit” your code and turn it into a PDF
- You need LaTeX to knit into a PDF

R Projects

- We’ll talk about this more as we get closer to the Final Paper
- This is great for keeping things organized and replicable

Code Chunks in R Markdown

If you want to run R code (or python!) inside a markdown, use this formatting

```
# this is a code chunk. Code is executed here. Functions done here are displayed in the knitted document
```

```
#Example
```

```
2+2
```

```
## [1] 4
```

First Steps

Text and code chunks:

```
# Text
```

```
print("Hello world")
```

```
## [1] "Hello world"
```

```
# Code
```

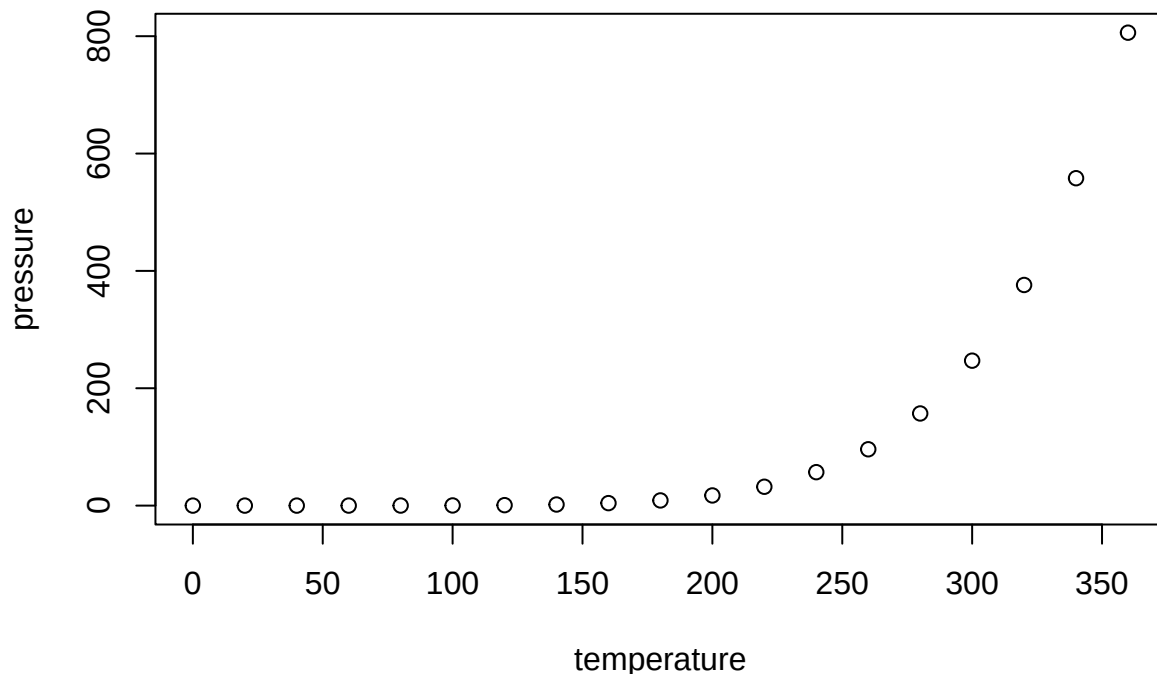
```
a <- 2 + 5
```

```
a
```

```
## [1] 7
```

```
6*a
```

```
## [1] 42
```



Some basics

Packages

- There are thousands of R packages that help you do things you want to do. They are basically pre-written functions that execute complex (sometimes) series of code and calculations given some pre-determined number of inputs.
- We will DEFINITELY be using the following packages: **tidyverse**, **estimatr**, **cobalt**, and **stargazer**. Let's install these together now.

How do you install new packages?

Generally speaking, will do this in console below, not in R script

install.packages('tidyverse')

install.packages('stargazer')

install.packages('estimatr')

install.packages('cobalt')

#install.packages('car')

Please delete the above lines of code once you have already installed the packages

What is downloaded and available to be used?

`library()`

`library(tidyverse)`

-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --

v dplyr 1.1.4 v readr 2.1.5

v forcats 1.0.0 v stringr 1.5.1

v ggplot2 3.5.2 v tibble 3.2.1

v lubridate 1.9.4 v tidyr 1.3.1

v purrr 1.0.4

-- Conflicts ----- tidyverse_conflicts() --

```
## x dplyr::filter() masks stats::filter()
## x dplyr::lag() masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

library(stargazer)

##
## Please cite as:
##
## Hlavac, Marek (2022). stargazer: Well-Formatted Regression and Summary Statistics Tables.
## R package version 5.2.3. https://CRAN.R-project.org/package=stargazer

library(estimatr)
library(cobalt)

## cobalt (Version 4.5.5, Build Date: 2024-04-02)
```

Getting Help

- If there is one skill that is most important to get good at quickly, it is learning where to look for help when you get stuck (you will... a lot).
- One of the best things about R is that there is such a broad user base and very active (don't ask me why) online message boards with help. The trick is knowing how to find it by knowing what to search for.
- Getting started: Using the `?` or `help()` The first place to start when you're using a package is getting used to looking at the help files. You can request the help file for any function in any package (help files are a requirement for publishing an R package)
- ChatGPT can be helpful but sometimes makes mistakes or suggests incorrect codes

```
help(sum)
?sum
```

- The key things to look at here are:
 - 1) Usage: what inputs does the function need;
 - 2) Arguments, what are the format of these inputs in more details;
 - 3) Value: what will the function return; and
 - 4) Examples: I often jump straight to the examples
- Getting help also means figuring out what is wrong when you get error messages. The best thing to do is to copy the message and search for it in google.

Basic Commands in R

- R is an object based language
- `<-` is an assignment operator. The single `=` is also an assignment operator for special uses that we will discuss later.
 - On the left is the variable name of the object where I am storing things
 - To the right is whatever I want to put into my object (You can never do the other way around!)

Math!

You can use R as a calculator

```
# Addition
5 + 6
```

```
## [1] 11
# Subtraction
9 - 3
```

```
## [1] 6
# Multiplication
5*8
```

```
## [1] 40
# Division
6/10
```

```
## [1] 0.6
# Exponentiation
5^2
```

```
## [1] 25
# Square Roots
sqrt(25)
```

```
## [1] 5
# Etc etc etc
```

R can also evaluate logical queries:

```
# True or False
5 < 6
```

```
## [1] TRUE
6 < 5
```

```
## [1] FALSE
2 >= 0 #greater than or equal to
```

```
## [1] TRUE
10 == 10 #notice the two equal signs
```

```
## [1] TRUE
```

Variables

Variables are the workhorse of R. They store whatever you give them.

There are naming conventions for variables. Some people like to use snake case for variable names:

```
some_use_snake_case

other.people.use.periods #Not recommended

othersUseCamelCase
```

Variable names cannot have spaces. This is entirely a matter of preference, but try to keep it consistent.

Google style guide uses the periods. Many who use other programming languages will strongly urge you to avoid using the period because there are certain functions in R that require the period `is.matrix()` and

other languages, like Python and SQL rely on the period in important ways to define hierarchies in function and object calls.

If you want to use camelCase, Google style guide suggests using **CamelCase** with a capitalized first letter for functions, only, and **camelCase** with a lower case letter if you prefer, for variable names.

ALWAYS make sure your variable name is descriptive of what it is representing. Do not name your variables vague terms (like **var**, which is what I sometimes still do).

Decide among your teammates which convention of variable naming you will be using. STAY CONSISTENT!

Let's start by making a variable called **x**:

```
# Assign the variable x the value 10
x <- 10
x #print
```

```
## [1] 10
```

I will often try to avoid cluttering my scripts by having an entire line devoted to printing out the contents of an object as I have done above. I will often assign an object and then just highlight the x on that same line and print it to make sure it was assigned properly.

Please assign the variable **y** the value 15 and check if you did it correctly.

```
# Assign the variable y the value 15
y <- 15
y
```

```
## [1] 15
```

We can do math with variables too. Look what happens now when you add x and y

```
# Add x and y
x + y
```

```
## [1] 25
```

We can also create a new variable to store the added value.

```
# Store the sum of x and y to a new variable z
z <- x + y
z #print
```

```
## [1] 25
```

You can mix types

```
# Create a character string
x <- 'This is a character string'
x
```

```
## [1] "This is a character string"
```

Notice that the character strings are in quotes (either single or double work, just be consistent).

But be weary that you cannot do certain operations with mixed types

```
x + y #cannot perform this operation
```

What kind of variable is x, now?

```
# Find what class the variables are
class(x) #character
```

```
## [1] "character"
```

```
class(y) #numeric
```

```
## [1] "numeric"
```

So we have numeric and character strings, what else can we have?

```
# Create a boolean variable (True/False)
```

```
my_boolean_variable <- TRUE
```

Note the lack of quotations. This is not a character string.

```
# Check the class of the variable
```

```
class(my_boolean_variable) #logical
```

```
## [1] "logical"
```

So far all I have shown you is variables with one object in them:

```
# Find out the number of objects inside a variable
```

```
length(x)
```

```
## [1] 1
```

These are actually called scalars (1 item). But variables can hold many objects. These variables are called vectors.

```
# Create a vector of numbers 1-6
```

```
vector <- c(1, 2, 3, 4, 5, 6)
```

```
vector #print
```

```
## [1] 1 2 3 4 5 6
```

```
class(vector) #check class
```

```
## [1] "numeric"
```

```
length(vector) #check number of objects
```

```
## [1] 6
```

Notice that `c()` is a function that **combines** or **concatenates** objects together. Note that all the objects in a vector need to be the same class. Tell me what happens when we combine character and numeric objects together:

```
# Combine numeric and character objects together
```

```
vector2 <- c(1, 2, 3, 'dog')
```

```
vector2 #print
```

```
## [1] "1" "2" "3" "dog"
```

```
class(vector2) #check class
```

```
## [1] "character"
```

```
length(vector2) #check number of objects
```

```
## [1] 4
```

We can change the class of the objects

```
# Change `dog` to `NA`
```

```
vector2[vector2 == 'dog'] <- NA
```

```
vector2 #check if changed correctly
```

```
## [1] "1" "2" "3" NA
class(vector2) #check class

## [1] "character"
# Change character to numeric
vector2 <- as.numeric(vector2)
vector2 #notice the quotes are gone

## [1] 1 2 3 NA
class(vector2) #check class

## [1] "numeric"
```

Data Frames

The last format we'll learn is the data frame. This is how you will work with data almost all of the time you are doing statistical analyses. Think again of the spreadsheet where you have rows (observations) and columns (variables). Dataframes are different than matrices because they can hold different types of data. One column can be numeric, another character, and another boolean.

```
# Load `car` package if you need to
library(car)

## Loading required package: carData
##
## Attaching package: 'car'
## The following object is masked from 'package:dplyr':
##
##      recode
## The following object is masked from 'package:purrr':
##
##      some
# `mtcars` is a dataset that is stored in the `car` package
mtcars
```

	mpg	cyl	dis	hp	drat	wt	qsec	vs	am	gear	carb
## Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
## Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
## Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
## Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
## Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
## Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
## Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
## Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
## Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
## Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
## Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4
## Merc 450SE	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	3
## Merc 450SL	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	3
## Merc 450SLC	15.2	8	275.8	180	3.07	3.780	18.00	0	0	3	3
## Cadillac Fleetwood	10.4	8	472.0	205	2.93	5.250	17.98	0	0	3	4
## Lincoln Continental	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	4
## Chrysler Imperial	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	4

```
## Fiat 128      32.4  4  78.7  66 4.08 2.200 19.47  1  1   4   1
## Honda Civic  30.4  4  75.7  52 4.93 1.615 18.52  1  1   4   2
## Toyota Corolla 33.9  4  71.1  65 4.22 1.835 19.90  1  1   4   1
## Toyota Corona 21.5  4 120.1  97 3.70 2.465 20.01  1  0   3   1
## Dodge Challenger 15.5  8 318.0 150 2.76 3.520 16.87  0  0   3   2
## AMC Javelin  15.2  8 304.0 150 3.15 3.435 17.30  0  0   3   2
## Camaro Z28   13.3  8 350.0 245 3.73 3.840 15.41  0  0   3   4
## Pontiac Firebird 19.2  8 400.0 175 3.08 3.845 17.05  0  0   3   2
## Fiat X1-9     27.3  4  79.0  66 4.08 1.935 18.90  1  1   4   1
## Porsche 914-2 26.0  4 120.3  91 4.43 2.140 16.70  0  1   5   2
## Lotus Europa  30.4  4  95.1 113 3.77 1.513 16.90  1  1   5   2
## Ford Pantera L 15.8  8 351.0 264 4.22 3.170 14.50  0  1   5   4
## Ferrari Dino  19.7  6 145.0 175 3.62 2.770 15.50  0  1   5   6
## Maserati Bora  15.0  8 301.0 335 3.54 3.570 14.60  0  1   5   8
## Volvo 142E    21.4  4 121.0 109 4.11 2.780 18.60  1  1   4   2
```

```
# Create your own dataframe by reproducing `mtcars` dataset
my_Data_Frame <- mtcars
```

There are some helper functions in R to help you look at a dataframe.

```
# View the first 6 rows
head(my_Data_Frame)
```

```
##           mpg cyl  disp  hp drat   wt  qsec vs am gear carb
## Mazda RX4      21.0   6  160 110 3.90 2.620 16.46  0  1   4   4
## Mazda RX4 Wag  21.0   6  160 110 3.90 2.875 17.02  0  1   4   4
## Datsun 710     22.8   4  108  93 3.85 2.320 18.61  1  1   4   1
## Hornet 4 Drive 21.4   6  258 110 3.08 3.215 19.44  1  0   3   1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0   3   2
## Valiant        18.1   6  225 105 2.76 3.460 20.22  1  0   3   1
```

```
# View the last 6 rows
tail(my_Data_Frame)
```

```
##           mpg cyl  disp  hp drat   wt  qsec vs am gear carb
## Porsche 914-2  26.0   4 120.3  91 4.43 2.140 16.7  0  1   5   2
## Lotus Europa   30.4   4  95.1 113 3.77 1.513 16.9  1  1   5   2
## Ford Pantera L 15.8   8 351.0 264 4.22 3.170 14.5  0  1   5   4
## Ferrari Dino   19.7   6 145.0 175 3.62 2.770 15.5  0  1   5   6
## Maserati Bora   15.0   8 301.0 335 3.54 3.570 14.6  0  1   5   8
## Volvo 142E     21.4   4 121.0 109 4.11 2.780 18.6  1  1   4   2
```

```
# Display the structure of each variable in the dataframe
str(my_Data_Frame)
```

```
## 'data.frame':   32 obs. of  11 variables:
## $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
## $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
## $ disp: num  160 160 108 258 360 ...
## $ hp : num  110 110 93 110 175 105 245 62 95 123 ...
## $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
## $ wt : num  2.62 2.88 2.32 3.21 3.44 ...
## $ qsec: num  16.5 17 18.6 19.4 17 ...
## $ vs : num  0 0 1 1 0 1 0 1 1 1 ...
## $ am : num  1 1 1 0 0 0 0 0 0 0 ...
## $ gear: num  4 4 4 3 3 3 3 4 4 4 ...
```

```
## $ carb: num 4 4 1 1 2 1 4 2 2 4 ...
```

```
# Summarizes each variable in the dataframe
```

```
summary(my_Data_Frame)
```

```
##      mpg      cyl      disp      hp
## Min.   :10.40  Min.   :4.000  Min.   : 71.1  Min.   : 52.0
## 1st Qu.:15.43  1st Qu.:4.000  1st Qu.:120.8  1st Qu.: 96.5
## Median :19.20  Median :6.000  Median :196.3  Median :123.0
## Mean   :20.09  Mean   :6.188  Mean   :230.7  Mean   :146.7
## 3rd Qu.:22.80  3rd Qu.:8.000  3rd Qu.:326.0  3rd Qu.:180.0
## Max.   :33.90  Max.   :8.000  Max.   :472.0  Max.   :335.0
##      drat      wt      qsec      vs
## Min.   :2.760  Min.   :1.513  Min.   :14.50  Min.   :0.0000
## 1st Qu.:3.080  1st Qu.:2.581  1st Qu.:16.89  1st Qu.:0.0000
## Median :3.695  Median :3.325  Median :17.71  Median :0.0000
## Mean   :3.597  Mean   :3.217  Mean   :17.85  Mean   :0.4375
## 3rd Qu.:3.920  3rd Qu.:3.610  3rd Qu.:18.90  3rd Qu.:1.0000
## Max.   :4.930  Max.   :5.424  Max.   :22.90  Max.   :1.0000
##      am      gear      carb
## Min.   :0.0000  Min.   :3.000  Min.   :1.000
## 1st Qu.:0.0000  1st Qu.:3.000  1st Qu.:2.000
## Median :0.0000  Median :4.000  Median :2.000
## Mean   :0.4062  Mean   :3.688  Mean   :2.812
## 3rd Qu.:1.0000  3rd Qu.:4.000  3rd Qu.:4.000
## Max.   :1.0000  Max.   :5.000  Max.   :8.000
```

You can find the dimensions of Dataframe and the number of rows and columns as well!

```
# Number of rows
```

```
nrow(my_Data_Frame)
```

```
## [1] 32
```

```
# Number of columns
```

```
ncol(my_Data_Frame)
```

```
## [1] 11
```

```
# Dimensions
```

```
dim(my_Data_Frame)
```

```
## [1] 32 11
```

We will learn basic data manipulation next week! Let me know if you have any questions on what we have covered today!